

TheraFlow Clinical Platform

Clinical workflows, practice finance logic, and a documented security-remediation snapshot.

BUYER DILIGENCE PACKAGE

Evaluate the implemented software and its evidence before connecting clinical or financial vendors.

Clinical workspaces

EHR, scheduling, documentation, assistant, and heuristic PHI redaction.

Practice operations

Compensation calculations, treasury simulation, payroll preparation, and double-entry ledger logic.

Synthetic evaluation scope. Real patient PHI and live regulated operations require further engineering, agreements, and verification.

A source repository with inspectable assets

The package documents code and synthetic workflows. Commercial rights and ownership require transaction diligence.

Application source

React 19 / TypeScript / Vite SPA, Express 4 API, clinical tools, and Practice OS modules.

Data and authorization definitions

Two SQL migrations with table definitions, RLS policies, and clinical-note write-protection triggers.

Algorithms and adapters

Integer-cent compensation and money allocation logic, ledger validation, note templates, export formatters, and provider scaffolds.

Evaluation materials

Automated tests, synthetic corpora, buyer handoff, acquisition-readiness memo, and annotated historical screenshots.

Separate the application from its vendor rails

Schema and adapter code demonstrate design intent; a configured deployment must be verified end to end.

1. Browser application

Clinical UI, local/synthetic state, redaction rules, formatting tools, and browser media controls.

2. Express API

JWT checks, role restrictions, tenant checks, financial workflow routes, and signed audit records.

3. Persistence and vendors

PostgreSQL/Supabase schemas and selected SDK paths. Persistence wiring, vendor activation, and operating controls remain buyer work.

The browser redaction engine runs locally. Browser speech recognition may use browser-vendor services; local audio processing is not guaranteed.

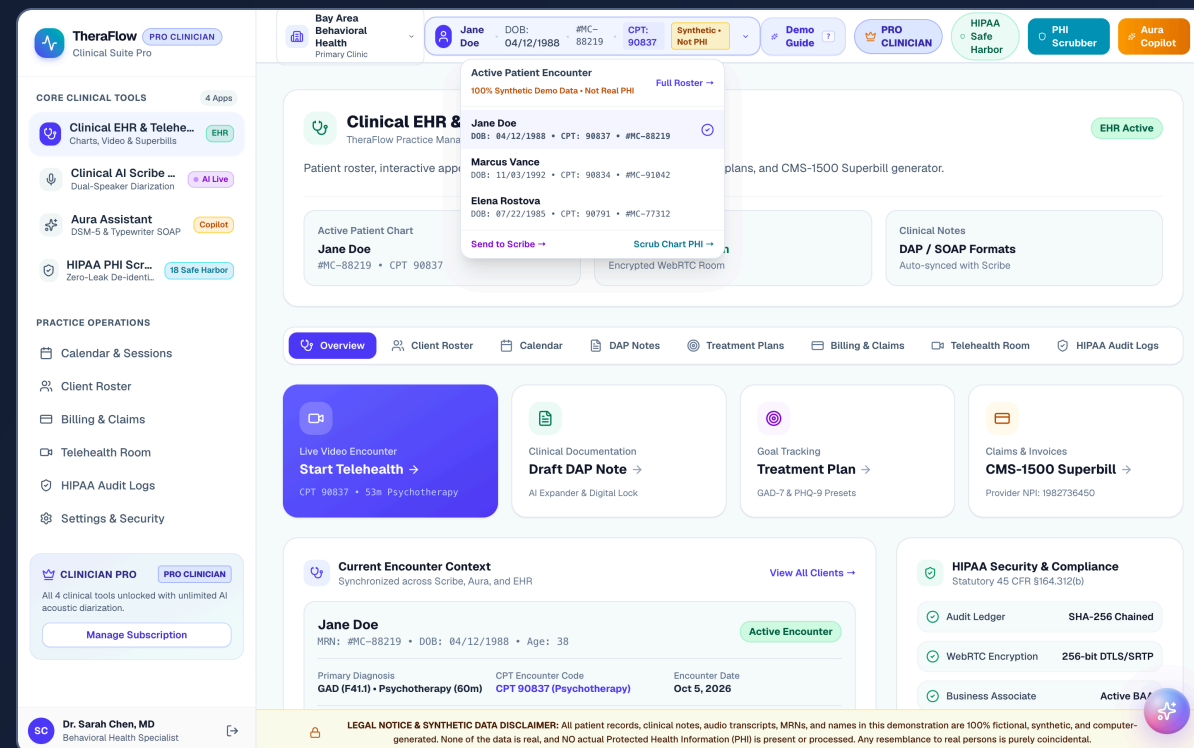
Source: package.json; server.ts; src/lib/; src/tools/; supabase/migrations/

Working code for the clinical workspace

These are implemented functions evaluated with synthetic records; test success does not establish clinical accuracy.

- Client roster, charts, scheduling, DAP notes, and treatment-plan UI.
- Clinical note templates with deterministic fallback and a configurable Gemini path.
- Aura criteria examples and note-assistance UI, requiring clinician review.
- Superbill reimbursement modal and text export; no implemented 837P claim generation or live clearinghouse submission.

Export adapters generate Epic/Cerner text and FHIR-shaped JSON for download or clipboard. They do not prove interoperability with a hospital EHR.



Historical synthetic chart screenshot; not recaptured for this accuracy release.

Clinical-to-financial workflow logic

Calculations and ledger invariants are real code. Bank transfers and tax filings are simulated or unfinished.

Compensation

Tiered revenue splits, CPT flat rates, bonuses, and payroll-preparation calculations.

Money allocation

Selected allocation paths use integer cents and basis points to conserve amounts within the tested bounds.

General ledger

Double-entry journal validation, balance checks, reversal logic, and tax-reserve bookkeeping.

Workflow cascade

Synthetic encounter-to-payment-to-compensation-to-payroll demonstrations with traceable ledger events.

No live bank accounts, ACH settlement, payroll tax filing, or real funds movement is established by these simulations.

Interactive demonstrations with explicit limits

Visible behavior must be distinguished from a connected external service.

Telehealth

Camera/microphone controls and WebRTC loopback. The meeting API returns simulated metadata; this is not a verified remote consultation service.

Speech and speaker labels

Synthetic utterances or browser SpeechRecognition with conversational heuristics. Acoustic diarization accuracy is not established.

Claims and invoice checkout

Claim statuses are synthetic UI data. Clearinghouse transmission and remittance ingestion are absent. Invoice checkout returns a simulated session rather than collecting a payment.

Subscriptions and treasury

Subscription checkout simulates when Stripe is unconfigured; the configured SDK path needs vendor testing. Treasury accounts and disbursements are synthetic.

Vendor activation needs engineering and verification

Adding credentials alone does not turn scaffolds into complete production integrations.

Remote telehealth and speech

Implement and verify session signaling, remote participant authorization, relay/media behavior, and a selected speech/diarization provider.

Insurance and EHR

Build authenticated clearinghouse transmission/remittance ingestion and hospital-authorized API connectivity. Validate generated formats with counterparties.

Payroll and banking

Complete commercial onboarding, provider-specific authentication, settlement/reconciliation, failure handling, and any tax-filing workflows.

Infrastructure and identity

Deploy both migrations, configure auth claims and secrets, verify persistence, backups, monitoring, and relevant vendor agreements.

Do not infer executed BAAs, vendor approval, regulatory certification, or operational launch readiness from source-level adapters.

Heuristic redaction is an aid with measured misses

Eighteen category definitions are not proof of complete identifier removal.

81.64%

Recorded earlier holdout recall

610 synthetic snippets / 3,410 annotated entities. 2,784 matched; 626 missed.

Source: phi_scrubber_holdout_results.json.

92.79%

Recorded fresh holdout recall

250 synthetic snippets / 1,540 annotated entities. 1,429 matched; 111 missed.

Source: THERAFLOW_REMEDIATION_PASS2_REPORT.md, Section M.

These are distinct historical synthetic corpora with overlap-based matching. Scores are not real-world recall estimates; recorded precision counts are not reconciled here.

The Scribe and DAP LLM paths invoke a sanitization gateway and abort on engine errors or known context name/MRN leftovers. Undetected identifiers can still pass. Clinician review and deployment privacy controls remain necessary.

SEC-01, SEC-02, and SEC-03 controls implemented

Core remediation reference: 4113573e982e6176ea8c71c3deb2862f44a6f11e.

SEC-01: API and tenant boundaries

Authentication and role checks on identified endpoints; cross-practice request tampering rejected; practice state and financial routes clamp access to the caller tenant.

SEC-02: Audit isolation

Legacy records are assigned to the demo practice instead of leaking across tenants. Audit writes, reads, and exports enforce practice boundaries.

SEC-03: Clinical-note writes

RLS and triggers restrict draft-note modification to its author or authorized practice administrators; signed-note mutations and author reassignment are blocked.

Evidence limits

Automated tests support the documented scenarios. They are not a third-party security certification, penetration test, HIPAA determination, or guarantee of zero vulnerabilities.

Source: server.ts; tests/server-security-and-tenant-isolation.test.ts; supabase/migrations/20261005_unified_practice_os.sql

31 public tables; 58 active RLS policies

Counts separate migration statements from the resulting catalog after both SQL files are applied.

31 public application tables

Unique public CREATE TABLE targets across the initial and remediation migrations.

32 total table targets

The 31 public tables plus auth.users, a standalone-PostgreSQL compatibility shim.

59 CREATE POLICY statements

Unique CREATE POLICY statements counted across both migration files.

58 active catalog policies

One broad clinical_notes policy is dropped and replaced by four granular policies. The final PostgreSQL catalog contains 58 policies.

Method: apply both migrations to a disposable PostgreSQL database and inspect `pg_tables` / `pg_policies`. Catalog counts establish installed assets, not an assessment of all authorization paths.

Source: supabase/migrations/20261005_init_schema.sql; supabase/migrations/20261005_unified_practice_os.sql; tests/migration-pipeline.test.ts

Passing suites on the immutable baseline snapshot

Verified snapshot runs: 37514914857 (master) and 37514932475 (remediation-pass-2).

SUITE	PASSING CHECKS IN SNAPSHOT LOGS
Client route/storage security	26 / 26
Server security / tenant isolation	29 / 29
Empirical server stress	28 / 28
EHR	30 / 30
Scribe	61 / 61
Aura / PHI scrubber	85 / 85
Financial / security invariants	Passing; inspect run logs

These historical runs did not execute the database migration test. Published v1.1.1 runs 37519919273 and 37519932172 include the original migration harness. The v1.1.2 candidate strengthens catalog/RLS assertions; use matching branch/PR CI evidence when completed.

[Master snapshot run](#) | [Remediation snapshot run](#) | [Published v1.1.1 evidence](#)

Preserve the baseline and resolve the corrected tag

v1.1.2-acquisition-accuracy is a release candidate. Preserve both existing published tags.

CORE SECURITY REMEDIATION

```
4113573e982e6176ea8c71c3deb2862f44a6f11e
```

IMMUTABLE BASELINE: V1.1.0-SECURITY-REMIATED

```
515cefe320b46b9e50695849dd23ce0c2b93cd63
```

PUBLISHED PACKAGE: V1.1.1-ACQUISITION-PACKAGE

```
fd9e2b3e442016f4dc9e85b893bb092b68393a97
```

CANDIDATE ACQUISITION ACCURACY RELEASE: V1.1.2-ACQUISITION-ACCURACY

```
git rev-parse v1.1.2-acquisition-accuracy^{commit}
```

The baseline annotated tag is unsigned. It is preserved; this package does not claim cryptographic signing or independent certification.

Candidate tag resolution succeeds only after publication. This PDF does not embed its enclosing commit SHA. Branch/PR CI evidence must identify the exact tested candidate commit before a new release is tagged.

[Source repository](#) | [Preserved published package](#)

Start with a synthetic-data evaluation

Node 22 and the documented configuration provide a starting point; validate the entire chosen hosting setup.

Install and build

Use the lockfile, run `npm ci`, `npm run typecheck`, and `npm run build`. Review server and SPA hosting requirements.

Identity and secrets

Set strong JWT and audit secrets; configure Supabase auth claims, tenant membership, and access rules. Verify demo-mode behavior is appropriate.

Persistence and operations

Apply both migrations and verify each persistence path, durable audit storage, backup/restore, alerting, and incident response.

Before patient or money flows

Move Gemini calls/secrets behind a server boundary: `vite.config.ts` embeds a configured key in client assets. Complete privacy/security, clinical, vendor-contract, and operating-control reviews.

The existing public demo URL is a discovery reference, not evidence that a frozen release is deployed or that patient PHI can be processed safely.

A reviewable acquisition snapshot

Software value rests on inspectable code, bounded evidence, and clear remaining work.

1. Read the handoff

BUYER_HANDOFF.md and ACQUISITION_READINESS.md define the functional boundaries and deployment work.

2. Reproduce the evidence

Follow release notes to the tested SHA and CI runs. Re-run synthetic suites and the PostgreSQL migration pipeline.

3. Inspect the workspaces

Use the walkthrough to identify workflows. Historical screenshots illustrate UI only and may contain older marketing language.

4. Scope the transaction

Verify ownership, licenses, dependencies, deployment design, integration effort, and commercial assumptions before purchase.

[Repository and current branch/PR evidence](#) | [Preserved published package](#)